

DOSSIER D'ARCHITECTURE LOGICIELLE

Orchestration BPMN via Flowable Engine (V2) - Extraits de Code

1. Philosophie de l'Orchestration

L'objectif de cette migration est de supprimer la logique de workflow codée en dur (algorithmes complexes de graphes) au profit d'un moteur certifié BPMN 2.0. Cela garantit une séparation stricte entre l'environnement de modélisation métier (Design Time) et le backend d'exécution (Runtime).

2. Base de Données : Jointure Conceptuelle

Pour lier le moteur Flowable aux entités métier, la classe Incident a été modifiée pour persister l'identifiant de l'instance du processus BPMN en cours.

src/.../model/Incident.java (Extrait)

```
@Entity
@Table(name = "incidents")
public class Incident {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "incident_code", unique = true)
    private String incidentCode;

    /**
     * Identifiant de l'instance BPMN dans le moteur Flowable.
     * Permet à l'API de retrouver la tâche active de cet incident.
     */
    @Column(name = "process_instance_id")
    private String processInstanceId;

    // ... autres propriétés
}
```

3. Backend (Java) : Démarrage du Processus via API Flowable

Lors de la création d'un incident, le service Java délègue la gestion de l'état au moteur RuntimeService de Flowable, au lieu de définir manuellement le statut.

src/.../service/IncidentService.java (Méthode createIncident)

```
java.util.Map<String, Object> variables = new java.util.HashMap<>();
variables.put("incidentCode", incident.getIncidentCode());

// Le moteur démarre l'instance basée sur le BPMN déployé
org.flowable.engine.runtime.ProcessInstance processInstance =
    runtimeService.startProcessInstanceByKey("incidentWorkflow", variables);

incident.setProcessInstanceId(processInstance.getId());

// Interrogation de la première tâche active définie par le workflow
org.flowable.task.api.Task currentTask = taskService.createTaskQuery()
    .processInstanceId(processInstance.getId())
    .singleResult();

if (currentTask != null) {
    incident.setStatus(currentTask.getName()); // ex: "Nouveau"
}
```

4. Backend (Java) : Déploiement Dynamique (Sans Recompilation)

Le point d'orgue de l'architecture est le déploiement "à chaud". L'API REST accepte un fichier XML généré depuis l'interface Modeler et l'injecte dans le moteur via le RepositoryService, mettant à jour la logique applicative en temps réel.

src/.../controller/WorkflowController.java (Extrait)

```
@PostMapping("/deploy-flowable")
public ResponseEntity<?> deployFlowableProcess(@RequestParam("file") MultipartFile file)
{
    try {
        org.flowable.engine.repository.Deployment deployment = repositoryService
            .createDeployment()
            .addInputStream(file.getOriginalFilename(), file.getInputStream())
            .name("Dynamic Deployment from UI")
            .deploy();

        return ResponseEntity.ok("Déploiement réussi");
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}
```

5. Frontend (React) : Alignement Fonctionnel

L'interface ReactFlow (V1) ayant été retirée, la vue Kanban s'aligne désormais strictement sur les 5 états définis formellement dans le modèle BPMN (Nouveau, Assigné, En cours, Résolu, Clôturé), prouvant que l'orchestration est maîtrisée centralement par le moteur.
